

Technical Report

Data Visualization Final Project

Social Network Visualization and Live Crypto Monitoring Dashboard

Field	Details
Group Members	WALEED AHMED AWADH AL-FAREH 230208818 ALI AHMED AWADH AL-FAREH 230208822 KHALED BA GERI 220208883 OSAMA MOHAMED AHMED 220208982
Track A	Game of Thrones Social Network Visualization & Community Discovery
Track B	Live Crypto Monitoring Dashboard
Main Tools	Python, Streamlit, NetworkX, PyVis, Plotly, Pandas, Requests
Deployment	Streamlit Community Cloud for Track B; source code hosted on GitHub

Submitted as part of the Data Visualization Final Project.

1. Executive Summary

This report documents a two-part data visualization project that satisfies the required Track A and Track B specifications. Track A transforms a relational character dataset into an interactive social network visualization. Track B builds a live cryptocurrency monitoring dashboard that uses a REST API when available and a demo fallback mode when the live API is blocked during presentation.

The main design goal across both tracks is glanceability: users should be able to identify key relationships, clusters, current status, and alerts quickly without being overloaded by unnecessary detail. Track A emphasizes structural audit and narrative, while Track B emphasizes velocity, temporal context, and alerting.

The project package includes organized source code, a README with execution instructions, CSV data files, dashboard scripts, screenshots or video evidence, and this technical report. The live dashboard deployment supports the Track B requirement for a functional live app or video recording of real-time behavior.

2. Visualization Design Document

2.1 User Persona

Primary user: A student, instructor, or project evaluator who needs to quickly understand whether the visualization communicates structure, importance, and temporal changes effectively.

User goals:

- Inspect social network communities without reading raw tables.
- Identify the most connected characters and bridge relationships.
- Monitor short-term cryptocurrency price movement in a simple live dashboard.
- Recognize threshold-based alerts immediately through color and status messages.

2.2 Visual Encoding Mapping

Track	Data Attribute	Visual Encoding	Purpose
Track A	Community group	Node color	Separates detected groups and makes clusters glanceable.
Track A	Degree centrality	Node size	Highlights characters with many direct relationships.
Track A	Relationship weight	Edge thickness	Communicates stronger and weaker ties.
Track A	Relationship type	Tooltip and optional edge label	Provides detail on demand while reducing clutter.
Track B	Current price	Metric card	Shows the current state clearly.
Track B	Recent price history	Line chart / sliding window	Communicates temporal movement and trend.
Track B	Threshold exceeded	Red alert message	Uses color to signal abnormal change.
Track B	Last updated time	Timestamp metric	Shows latency and confirms refresh behavior.

3. Track A - Social Network Visualization & Community Discovery

3.1 Data Preparation and Structural Mapping

Track A uses a Game of Thrones character relationship dataset. The data is represented through two CSV files: an edge list and a node table. The edge list defines source character, target character, relationship strength, and relationship type. The node table documents the available characters and can support additional attributes if expanded later.

Edge definition: An edge represents a relationship between two characters. Examples include Alliance, Friendship, Sibling, Advisor, Protector, and Conflict. Edge weight represents tie strength; higher weights are drawn as thicker edges.

Dataset File	Role in Visualization
track_a_social_network/data/edges.csv	Defines Source -> Target connections, relationship type, and weight.
track_a_social_network/data/nodes.csv	Defines the character nodes available in the network.

3.2 Community Detection

The project applies a modularity-based community detection method to partition the network into logical sub-groups. In the implementation, NetworkX is used for graph analysis, and a greedy modularity algorithm is suitable because it starts with each node as its own community and repeatedly merges communities that improve modularity until no further improvement is possible.

The resulting communities are represented with different node colors. In the Game of Thrones context, detected groups can be interpreted as narrative clusters such as Stark-related characters, Lannister-related characters, and Targaryen or alliance-related characters.

3.3 Key Actors and Centrality

The key actor metric used in the visualization is degree centrality. Degree centrality counts how many direct connections each node has. In this project, characters with more direct relationships are drawn larger, allowing users to identify central characters at a glance.

The dashboard summary panel lists the top actors. Based on the current dataset and dashboard output, examples of important actors include Jon Snow, Daenerys Targaryen, Sansa Stark, Tyrion Lannister, and Cersei Lannister.

3.4 Layout Justification

A force-directed layout is used because it naturally places highly connected nodes closer together while pushing unrelated nodes apart. This helps reveal clusters, bridge nodes, and relationship structures. The dashboard also allows users to zoom, drag, and inspect tooltips, which improves exploration compared with a static table.

To reduce visual noise, relationship labels are optional. They can be enabled for inspection or disabled to avoid clutter. Details remain available through tooltips, which supports the principle of showing detail on demand.

3.5 Track A Dashboard Screenshot



Figure 1. Track A social network dashboard showing metrics and the interactive network.

3.6 Community Narrative

The network reveals three broad communities in the current dataset. The Stark community includes family and ally relationships centered around characters such as Jon Snow, Sansa Stark, Arya Stark, and Bran Stark. The Lannister community includes relationships among Tyrion, Jaime, and Cersei, with conflict and family ties defining the structure. The Targaryen alliance community contains Daenerys and supporting characters such as Jorah, Missandei, Yara, and related allies.

The visual separation indicates that the narrative network is not random. Characters tend to group by family, political alliance, or story arc. Nodes that connect between groups are analytically important because they help explain how influence or conflict travels across the network.

3.7 Weak Link Analysis

A weak link is a relationship or character that connects otherwise separate groups. In this dataset, relationships such as Yara Greyjoy -> Daenerys Targaryen or Tyrion Lannister -> Jon Snow can act as bridge connections between story clusters. If such a bridge were removed, the network would become more fragmented, and the visual separation between communities would increase.

This analysis is important because bridges often represent narrative or strategic connections. Removing a bridge may not reduce the number of nodes, but it can change how groups are connected and how information, alliance, or conflict can move through the network.

3.8 Visualization Critique

One noisy area in network visualization is edge and label overlap. As the dataset grows, displaying every relationship label can create a hairball effect. The project reduces this issue by making relationship labels optional and keeping relationship details in tooltips. If more time were available, the next improvement would be label proportionality: only the top 3-5 most important nodes would show labels by default, while other labels would appear on hover.

4. Track B - Real-Time Data Visualization Dashboard

4.1 Data Pipeline Architecture

Track B is a live crypto monitoring dashboard built with Streamlit. The dashboard uses CoinGecko API when available and uses demo fallback data if the API is blocked. The API integration uses REST polling, meaning the dashboard periodically requests the latest price data rather than receiving pushed WebSocket updates.

Pipeline Stage	Implementation
Source	CoinGecko API simple price endpoint when available.
Ingestion method	REST polling through the Python requests library.
Refresh frequency	User-controlled interval from the sidebar, typically 10-60 seconds.
Storage	Recent readings stored in Streamlit session state as a sliding window.
Visualization	Metric cards, timestamp, connection status, Plotly line chart, and alert banner.

4.2 Temporal Window and Dynamic Visuals

The dashboard maintains a sliding window of recent price observations. This design is more useful than showing a single number because it provides historical context for the current price. Users can see whether the current value is stable, rising, or falling relative to recent readings.

The primary visual is a line chart that plots price against time. A small number of recent observations is kept to reduce cognitive load and make the trend readable. The current price, 24-hour change, window change, and last update time are shown as metrics above the chart.

4.3 Alerting and Threshold Logic

The dashboard includes a threshold slider. When the absolute window change percentage exceeds the selected threshold, the dashboard displays a red alert message. If the value remains within the allowed range, a green status message is displayed. This conditional formatting supports fast anomaly detection.

For presentation, the threshold can be lowered to demonstrate the alert. The fallback mode can also generate simulated movement if the live API is blocked, ensuring the dashboard remains usable during demonstration.

4.4 Track B Dashboard Screenshot



Figure 2. Track B live crypto monitoring dashboard with metrics, status, and time-series chart.

4.5 Performance and Latency

The dashboard uses a user-controlled refresh interval to balance real-time responsiveness with API rate limits and presentation stability. A 10-60 second interval is appropriate for a student project because it is frequent enough to show movement but conservative enough to avoid unnecessary API calls.

Latency is communicated through the Last Updated timestamp. This helps users understand when the dashboard last refreshed and whether the view is current. If the API fails, the connection metric changes to Demo Fallback, making the system state explicit.

4.6 User Experience

The dashboard uses metric cards for essential numbers and a line chart for temporal context. Bright warning color is reserved for alert conditions, while normal operation is communicated with a green status message. This reduces cognitive load by keeping the interface calm unless action is needed.

The sidebar contains configuration controls such as coin ID, currency, refresh interval, threshold, auto-refresh, and demo fallback. Moving controls to the sidebar keeps the main dashboard focused on monitoring.

4.7 Future Scaling

If the data source suddenly produced 1,000 updates per second, the current Streamlit session-state approach would become inefficient. The first bottleneck would likely be chart rendering and repeated reruns. A scalable design would use a message queue or streaming database, aggregate data into time buckets, and update the chart at a lower visual frame rate. This would preserve responsiveness while avoiding visual overload.

5. Implementation Summary

Component	File / Tool	Purpose
Prepared for Data Visualization Final Project		

Track A script	track_a_social_network/ social_network_analysis.py	Builds graph, applies community detection, and renders interactive network.
Track A data	track_a_social_network/data/ edges.csv and nodes.csv	Stores relationship and node data.
Track B script	track_b_realtime_dashboard/ crypto_dashboard.py	Creates crypto monitoring dashboard with API/fallback, chart, metrics, and alert logic.
Dependencies	requirements.txt	Lists Python packages required for local and cloud execution.
Documentation	README.md	Explains project structure, setup, and run commands.

6. Conclusion

The final project satisfies the core intent of both required tracks. Track A provides a structural audit of a social network through community detection, centrality-based visual hierarchy, and interaction. Track B provides temporal monitoring through a live-style dashboard with REST API integration, sliding-window charting, status timestamps, and threshold-based alerting.

Together, the two tracks demonstrate how visual design can reduce cognitive load: Track A makes network structure easier to interpret, while Track B makes real-time changes and anomalies easier to notice.

7. References

1. Final Project - Data Visualisation brief provided by instructor.
2. CoinGecko API Documentation - Simple Price endpoint: <https://docs.coingecko.com/reference/simple-price>
3. Streamlit Community Cloud dependencies documentation: <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/app-dependencies>
4. NetworkX greedy_modularity_communities documentation: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.modularity_max.greedy_modularity_communities.html
5. PyVis documentation: <https://pyvis.readthedocs.io/en/latest/documentation.html>